AI INTERVIEW QUESTION GENERATOR

Project Report

Remote Internship — Task 4

Submitted by: Alishba

Python | Pandas | Hugging Face Transformers | FLAN-T5 Small

1. Introduction

The AI Interview Question Generator is a Python-based project designed to assist in preparing interview questions for internship candidates. The system uses candidate profiles, job descriptions, and an existing question bank to produce technical and behavioral interview questions.

The system allows users to select a candidate, choose a job role, and specify an interview difficulty level. It also analyzes the candidate's skills against the skills required for the selected job.

The main purpose of this project is to make interview preparation more organized and provide questions that consider the candidate's background and the requirements of the internship position.

2. Project Objectives

The main objectives of this project are:

- To generate technical and behavioral interview questions for internship candidates.
- To use candidate profiles and job descriptions as input.
- To support multiple interview difficulty levels.
- To identify skills a candidate already possesses and skills they may need to develop.
- To integrate a text-generation model into the question-generation workflow.
- To provide a structured set of interview questions for preparation and evaluation.

3. Technologies and Tools

The following technologies and tools were used in the project:

Tool	Purpose
Python	Main programming language.
Pandas	Used to load and process CSV datasets.
Hugging Face Transformers	Used to integrate the text-generation model.
Google FLAN-T5 Small	Pre-trained text-to-text model integrated into the system.
Visual Studio Code	Development environment.
CSV files	Used to store candidate profiles, job descriptions, and interview questions.

3.1 Model Selection: Why Google FLAN-T5 Small?

The project instructions mentioned GPT-3 and LLaMA as examples of text-generation models. However, this project uses Google's FLAN-T5 Small, a pre-trained text-to-text model available through Hugging Face. The following factors influenced this choice:

1. Free and Accessible

FLAN-T5 Small is available as a pre-trained model through Hugging Face, making it accessible for student projects without requiring a paid API subscription.

2. No Paid API Key Required

Unlike API-based access to GPT-3, the model can be downloaded and run locally without requiring an OpenAI API key or paying per API request.

3. Suitable for a Student Project

FLAN-T5 Small provides a practical way to experiment with text generation and natural language processing using Python.

4. Lower Computational Requirements

The Small variant has fewer parameters than larger language models, making it a more practical option for a computer with limited computational resources.

5. Integration with Python

The model integrates with the Hugging Face Transformers library, allowing it to be loaded and used directly in the Python application.

6. Text-to-Text Architecture

FLAN-T5 takes text as input and generates text as output. This makes its architecture suitable for tasks involving instructions, prompts, and question generation.

7. Practical for Experimentation

Using FLAN-T5 Small allowed the project to incorporate a pre-trained language model without needing to train a large model from scratch.

Comparison with GPT-3 and LLaMA

Factor	FLAN-T5 Small	GPT-3	LLaMA
Access	Available through Hugging Face	Commonly accessed through an API	Availability depends on the specific version and license
Cost	Can be used without a paid API subscription	API usage may incur charges	Some versions can be used without API charges
Local execution	Possible on suitable hardware	Generally accessed through an API	Possible with suitable hardware and model configuration
Hardware requirements	Relatively low	Managed by the API provider	Vary significantly by model size
Python integration	Hugging Face Transformers	API integration	Available through various inference tools
Suitability for this project	Practical for a lightweight student project	An alternative requiring API access	An alternative requiring model setup and potentially more resources

Important Consideration

FLAN-T5 Small was selected primarily for accessibility, ease of integration, and lower computational requirements. However, smaller models may produce less reliable or less relevant questions than more capable language models.

In this project, the model is integrated into the question-generation workflow, while existing question-bank questions provide a fallback when generated output is unsuitable. Therefore, the final output does not depend entirely on the model generating every question correctly.

Justification of Choice

I selected Google FLAN-T5 Small instead of GPT-3 or LLaMA because it is lightweight, freely accessible, and easier to run within my system's computational limitations. GPT-3 generally requires API access, while larger LLaMA models may require more computing resources. FLAN-T5 Small allowed me to integrate a pre-trained text-generation model into my project without additional API costs.

4. Datasets

The project uses three CSV datasets.

4.1 Question Bank

File: question_bank.csv

The question bank contains existing interview questions organized by difficulty level. It provides a collection of questions that the system can select from during the interview-question generation process. The difficulty categories used in the project are Easy, Medium, and Hard.

4.2 Intern Profiles

File: intern_profiles.csv

This dataset contains information about internship candidates, including:

- Candidate ID
- Educational background
- Technical skills
- Projects
- Experience level

The system uses the selected candidate's profile to provide context for the interview questions.

4.3 Job Descriptions

File: job_descriptions.csv

This dataset contains information about available internship roles, including:

- Job ID
- Job title
- Required skills
- Job responsibilities

The selected job description helps the system identify the skills required for a particular internship and compare them with the candidate's existing skills.

5. Project Implementation: Major Steps

Step 1: Importing Required Libraries

The project begins by importing the required Python libraries. Pandas is used for handling the CSV datasets, while the Hugging Face Transformers library provides the components needed to load and use the pre-trained text-generation model.

Step 2: Loading the Datasets

The program loads the three CSV files (question_bank.csv, intern_profiles.csv, and job_descriptions.csv). These datasets provide the information required for question selection, candidate analysis, and job-role understanding.

Step 3: Selecting the Candidate

The user selects a candidate using their unique candidate ID. The program retrieves the candidate's educational background, technical skills, projects, and experience level from the intern profiles dataset. This information provides context for preparing interview questions.

Step 4: Selecting the Job Role

The user selects a job using its unique job ID. The program retrieves the job title, required skills, and responsibilities from the job descriptions dataset. This allows the system to identify the technical requirements associated with the selected internship position.

Step 5: Selecting the Difficulty Level

The program allows the user to select one of three difficulty levels: Easy, Medium, or Hard. The selected level determines which difficulty category of the existing question bank is used. This feature allows the system to support candidates with different experience levels and interview preparation needs.

Step 6: Analyzing Candidate Skills

The system compares the candidate's existing skills with the skills required by the selected job. The skills are divided into two categories: **matched skills** (skills that appear in both the candidate profile and the job requirements) and **missing or learning-area skills** (required skills that are not listed in the candidate's profile). For example, when candidate INT009 is selected for the NLP Intern position, Python is identified as a matched skill, while NLP, Scikit-learn, Transformers, and Text Processing are identified as learning areas. This analysis highlights potential areas for further learning and interview preparation.

Step 7: Loading the Text-Generation Model

The project integrates the pre-trained google/flan-t5-small model through the Hugging Face Transformers library. The model is loaded using its tokenizer and sequence-to-sequence model components. The system attempts to use the model in the question-generation workflow. Existing question-bank questions are also used as a fallback when generated output is unsuitable. This combination helps the program continue producing questions even when model-generated output is not appropriate.

Step 8: Generating Technical Questions

The program prepares technical interview questions using the selected difficulty category and the available question bank, with the text-generation component integrated into the workflow. Technical questions assess a candidate's programming knowledge, technical fundamentals, and understanding of concepts relevant to internship work. The program displays five technical questions for the selected candidate and job.

Step 9: Generating Behavioral Questions

The program also prepares behavioral interview questions. These questions focus on the candidate's project experience, motivation, willingness to learn, problem-solving approach, and ability to manage responsibilities. The current implementation uses predefined question templates to produce behavioral questions based on the selected candidate's project and job role. Five behavioral questions are displayed for each interview session.

Step 10: Displaying the Results

After generating the questions, the program displays the complete interview question set in the terminal. The output includes:

- Candidate information
- Job description
- Matched skills
- Missing or learning-area skills
- Five technical interview questions
- Five behavioral interview questions
- A summary of the selected candidate, job role, difficulty level, and question counts

This structured output makes the results easy to read and use for interview preparation.

6. System Workflow

The overall workflow of the project is:

1. Start the Python program.
2. Load the three CSV datasets.
3. Select an intern profile.
4. Select a job description.
5. Choose the interview difficulty level.
6. Retrieve the candidate's information.
7. Retrieve the job's requirements.
8. Compare candidate skills with required job skills.
9. Load and use the text-generation model.
10. Prepare technical questions using the question bank and generation workflow.
11. Prepare behavioral questions using predefined templates.
12. Display the questions and final summary.

7. Sample Execution and Results

The system was tested using the following example.

Parameter	Value
Selected candidate	INT009
Education	BS Software Engineering

Parameter	Value
Skills	Python, SQL, Testing, Git
Project	Software Testing Automation Project
Experience	Beginner
Selected job	NLP Intern
Required skills	Python, NLP, Scikit-learn, Transformers, Text Processing
Difficulty level	Easy

Skill Analysis

The system identified **Python** as a matched skill. The following skills were identified as missing or learning areas:

- NLP
- Scikit-learn
- Transformers
- Text Processing

Generated Interview Questions

The program successfully displayed five technical questions and five behavioral questions. The technical questions covered topics including Git, training and testing datasets, supervised learning, classification and regression, and Python data structures. The behavioral questions covered project experience, interest in the selected internship, willingness to learn, handling mistakes, and managing deadlines.

The program completed execution and displayed a final summary confirming the selected candidate, job role, difficulty level, and total number of questions.

Observed result: The system successfully loaded the datasets, performed skill analysis, loaded the text-generation model, and displayed ten interview questions.

8. Key Features

The main features of the project include:

Feature	Description
Candidate selection	Supports multiple intern profiles.
Job selection	Supports multiple internship descriptions.
Difficulty selection	Provides Easy, Medium, and Hard categories.
Skill-gap analysis	Identifies matched and missing skills.
Technical question preparation	Uses an existing question bank and integrates a text-generation model into the workflow.
Behavioral question preparation	Uses predefined templates to produce behavioral questions.

Feature	Description
Structured output	Displays questions and a final execution summary.

9. Limitations

Although the project runs successfully, several limitations should be considered.

1. **Question relevance:** Some technical questions may be general rather than directly related to the selected job's specialized skills.
2. **Model-generated output:** A pre-trained text-generation model can sometimes produce unsuitable questions. The implementation uses question-bank questions as a fallback when necessary.
3. **Behavioral question generation:** Behavioral questions currently rely on predefined templates rather than being generated entirely by the language model.
4. **Skill matching:** Skill-gap analysis depends on the skills recorded in the candidate profile and job description. A skill that is not listed in the profile may still be known to the candidate.
5. **Evaluation:** The project has been tested for successful execution and question generation, but a formal evaluation of question relevance and quality has not been conducted.

These limitations provide opportunities for future improvement.

10. Future Enhancements

Possible future enhancements include:

- Improving the relevance of questions to each job role.
- Expanding the question bank with more technical and behavioral questions.
- Improving the model prompts to produce more reliable questions.
- Generating behavioral questions dynamically using a language model.
- Adding a graphical user interface.
- Allowing users to export interview questions to a file.
- Evaluating question quality across different candidates, job roles, and difficulty levels.

11. Conclusion

The AI Interview Question Generator demonstrates how Python, structured CSV datasets, and a pre-trained text-generation model can be combined to support internship interview preparation.

The system allows users to select candidates, choose job roles, select difficulty levels, analyze matched and missing skills, and obtain technical and behavioral interview questions.

The successful execution of the program demonstrates that the main workflow is operational. Although question relevance and model output quality can be improved, the project provides a working foundation for a more advanced interview-question generation system.

Overall, this project provides practical experience in data processing, text generation, skill analysis, and the development of an AI-assisted application.

Submitted by: Alishba